Fundamentals of Programming using Python • First Year, FinTech

Chapter 04: Numbers, Strings and Booleans

`int` and `float` • the float trap • indexing and slicing • string methods • formatting • `bool` and truthiness

Chapter 04 of 10

Dr. Brahim Zirari

Yahia Farès University of Medea

in linkedin.com/in/brahimzirari · Google Scholar

October 2026

1. The three types

❑ **Where we are** – Chapter 01 named four types in passing. This chapter is the detail: what each one can hold, how they convert, and the traps in each.

2. Integers

❑ **No limit** – A Python `int` is a whole number of **any** size. Unlike most languages there is no maximum: the only limit is the memory of the machine.

```
print(2 ** 100)
print(len(str(2 ** 1000)))
```

Output

```
1267650600228229401496703205376
302
```

❑ **Underscores in long numbers** – Python ignores underscores inside a number literal. Use them: a figure in dinars is far easier to check.

```
capital = 1_250_000
print(capital)
```

Output

```
1250000
```

Remark: the underscores exist only in your file. They are not stored and not printed. `1_250_000` and `1250000` are the same value — only one of them can be read at a glance.

3. Floats, and the trap

❑ **What a float is** – A `float` is a decimal number, stored in binary with about 15–17 significant digits. Numbers such as 0.1, 0.2 and 0.035 **cannot be written exactly in binary**, just as 1/3 cannot be written exactly in decimal. What is stored is the nearest value the machine can hold.

Pitfall: this is not a Python fault and not a bug. Every language using the same standard behaves identically. Two consequences for finance work: never show a raw float to a human — `round()` or an f-string first — and never expect two floats computed by different routes to match to the last digit.

❑ **`round()` with a second argument** – `round(x, n)` keeps `n` decimals. A *negative* `n` rounds to the left of the point, which is how you report a figure in thousands.

```
print(round(1487.4567, 2))
print(round(1487.4567))
print(round(1487.4567, -2))
print(round(1487.4567, -3))
```

Output

```
1487.46
1487
1500.0
1000.0
```

Remark: `round(x)` with no second argument returns an `int`; `round(x, n)` returns a `float`, even when `n` is negative. That is why the last two lines print `1500.0` and not `1500`.

❑ **Converting** – `int` cuts the decimal part off — it does **not** round, and it cuts *towards zero*.

```
print(int(7.9), int(7.1), int(-7.9))
print(round(7.9), round(7.1), round(-7.9))
```

Output

```
7 7 -7
8 7 -8
```

Pitfall: using `int()` where you meant `round()` is a silent error: it produces a plausible number that is quietly too small. In a report of unit counts it loses one unit in nearly every row.

4. Two more numeric tools

❑ **`divmod`** – `divmod(a, b)` returns the whole part and the remainder together — the `//` and `%` of Chapter 01 in a single call.

```
print(divmod(7450, 4))
print(abs(-1250), abs(1250))
```

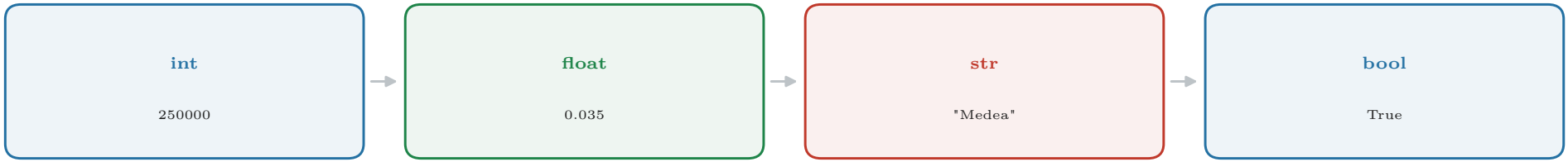


Figure 1. The types this chapter covers. Every value you write belongs to exactly one of them.

Output

```
(1862, 2)
1250 1250
```

Remark: the brackets in `(1862, 2)` mark a tuple*, a pair of values travelling together. Chapter 06 covers them; for now, read it as "both answers at once".*

❑ **Complex numbers** – Python has a fourth numeric type, `complex`, written `3 + 4j`. It exists, it is used in engineering and signal work, and you will not need it in this course. It is mentioned so that `<class 'complex'>` in an error message does not puzzle you.

5. Strings

❑ **Three ways to quote** – Single and double quotes are identical in Python; choose whichever avoids escaping. Triple quotes span several lines.

```
bank = 'BNA'
city = "Medea"
quote = "the client said \"no\""
address = """12 rue Didouche Mourad
Medea 26000"""
print(quote)
print(address)
```

Output

```
the client said "no"
12 rue Didouche Mourad
Medea 26000
```

❑ **Joining and repeating** – `+` joins two strings; `*` repeats one. Both need *both* sides to be the right type.

```
print("DZ" + "1234")
print("-" * 30)
# print("Total: " + 1250)   <- TypeError
```

Output

```
DZ1234
------------------------------
```

Pitfall: `"Total: " + 1250` is a `TypeError`: Python refuses to guess whether you meant to join or to add. Either convert — `"Total: " + str(1250)` — or, better, use an f-string.

6. Indexing and slicing

❑ **Each character has a position** – Counting starts at **0**. A negative index counts back from the end, so `-1` is the last character. `len` gives the number of characters.

```
s = "DZ1234567"
print(len(s))
print(s[0], s[1], s[-1])
print(s[0:2], s[2:], s[-3:])
```

Output

```
9
D Z 7
DZ 1234567 567
```

❑ **The slice rule** – `s[a:b]` takes the characters from position `a` **up to but not including** `b`. Leaving out `a` means "from the start"; leaving out `b` means "to the end".

$$\boxed{\text{length of } s[a{:}b] \;=\; b - a}$$

where a is the first position taken and b the first position *not* taken. For `s[2:5]` that is $5 - 2 = 3$ characters. This subtraction only works because the end is excluded — which is the reason Python excludes it.

Pitfall: an index past the end is an `IndexError`, but a slice *past the end is not: `s[0:99]` quietly returns the whole string. A slice never raises, so a wrong slice gives you a wrong answer in silence.*

7. String methods

❑ **What a method is** – A **method** is a function that belongs to a value, called with a dot: `value.method()`. Strings have about 45 of them; eight cover most of the work.

```
raw = "  bna - medea  "
print(raw.strip())
print(raw.strip().upper())
print(raw.strip().title())
print(raw.replace("-", "/").strip())
print("DZ1234".startswith("DZ"))
print("medea".find("d"))
```

Output

```
bna - medea
BNA - MEDEA
Bna - Medea
bna / medea
True
2
```

*Pitfall: **a string method never changes the string.** Strings are* immutable*: `raw.strip()` returns a new string and leaves `raw` exactly as it was. If you want to keep the result you must assign it: `raw = raw.strip()`. Forgetting this is the commonest string mistake there is.*

Remark: methods chain left to right. `raw.strip().upper()` strips first, then capitalises the result. Each call hands its new string to the next.

8. Formatting numbers for a report

❑ **The f-string mini-language** – After a colon inside `{}` you may say how the value should look.

Form	Result for 1487.4567	Use
`{x}`	1487.4567	raw; almost never right
`{x:.2f}`	1487.46	money, always
`{x:.0f}`	1487	whole units
`{x:,.2f}`	1,487.46	thousands separated
`{x:>12}`	1487.4567	right-aligned in 12 columns
`{x:<12}`	1487.4567	left-aligned
`{x:+.2f}`	+1487.46	always show the sign

```
amount = 1487.4567
print(f"{amount:.2f}")
print(f"{amount:,.2f}")
print(f"{amount:>12,.2f}|")
print(f"{-amount:+.2f}")
```

Output

```
1487.46
1,487.46
    1,487.46|
-1487.46
```

Remark: right-alignment in a fixed width is what makes a column of figures readable: the decimal points line up. It is the one formatting choice that matters in a financial report.

9. Booleans

❑ **Two values** – A `bool` is `True` or `False` — capitalised, with no quotes. `"True"` with quotes is a string, and a different thing entirely.

❑ **Truthiness** – `bool(x)` converts any value to a boolean. The rule is short: **empty and zero are `False`, everything else is `True`.**

```
print(bool(0), bool(0.0), bool(""))
print(bool(1), bool(-1), bool(0.1))
print(bool("0"), bool(" "), bool("False"))
```

Output

```
False False False
True True True
True True True
```

Pitfall: the last line is the trap. `"0"`, a single space, and the text `"False"` are all non-empty strings, so all three are `True`. A value read with `input()` is a string, so testing it directly tests only whether the user typed anything.

❑ **Booleans are numbers** – `True` equals 1 and `False` equals 0, and they may be added. Chapter 09 uses this to count things.

```
print(True + True + False)
print(int(True), float(False))
```

Output

```
2
1 0.0
```

10. The three types side by side

	`int` / `float`	`str`	`bool`
Written	`250000`, `0.035`	`"Medea"`	`True`
`+` means	add	join	add as 1 and 0
`*` means	multiply	repeat	—
Has a length	no	yes, `len()`	no
Indexable	no	yes, from 0	no
Changeable	no — all three are immutable; an operation returns a *new* value		

Quick reference

Numbers

`1_250_000`	underscores are ignored; use them
`round(x, 2)`	2 decimals, returns a float
`round(x)`	nearest whole number, returns an int
`round(x, -3)`	nearest thousand
`int(7.9)`	`7` — cuts towards zero, does not round
`divmod(a, b)`	the pair `(a // b, a % b)`
`abs(x)`	drop the sign

Strings

`len(s)`	number of characters
`s[0] s[-1]`	first, last
`s[a:b]`	from `a`, up to but not including `b`
`s[:b] s[a:]`	from the start / to the end
`s + t s * 3`	join, repeat
`.strip() .upper()`	and `.lower() .title()`
`.replace(a, b)`	every `a` becomes `b`
`.count(x) .find(x)`	how many; where, or `-1`
immutable	a method returns a new string. Assign it

Formatting

`{x:.2f}`	two decimals — money
`{x:,.2f}`	with thousands separators
`{x:>12}`	right-aligned in 12 columns

Booleans

`True False`	capitalised, no quotes
falsy	`0`, `0.0`, `""`, `None`
truthy	everything else, including `"0"` and `" "`
`True + True`	`2`

What you can do after this chapter

- Explain why `0.1 + 0.2` is not `0.3`, and what to do about it.
- Choose correctly between `int()` and `round()`.
- Index and slice a string, and compute the length of a slice.
- Use the eight string methods, and say why the original string is unchanged.
- Format a number for a report: decimals, thousands, alignment.
- Say which values are falsy, and why `"0"` is not one of them.

Two facts about floats that cost students marks every year.

```
>>> 0.1 + 0.2
0.30000000000000004
>>> round(0.1 + 0.2, 2)
0.3
>>> int(7.9)
7      <- cut, not rounded
>>> round(7.9)
8
```

Figure 2. The two float results that surprise everyone the first time.

❑ **Next** – Chapter 05 — the first container: lists.

↪ floating point explained: docs.python.org/3/tutorial/floatingpoint.html

↪ all string methods: docs.python.org/3/library/stdtypes.html

↪ the format mini-language: docs.python.org/3/library/string.html

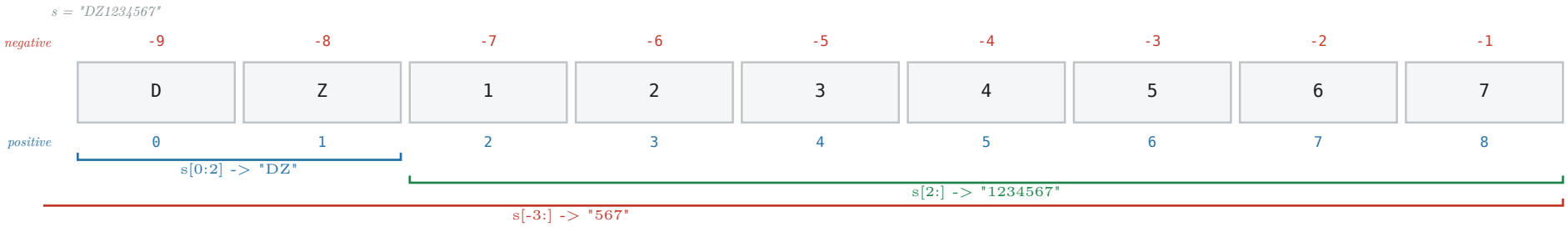


Figure 3. One string, two numbering systems, three slices.

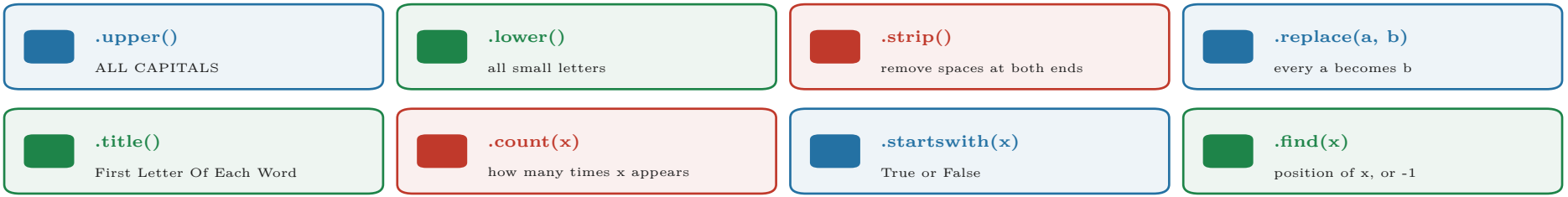


Figure 4. The eight string methods worth knowing now.